

```
int[] arr = new int[5]
```

By default values $\Rightarrow$ arr = {0, 0, 0, 0, 0}

```
String[] arr = new String[5]
```

By default values $\Rightarrow$ arr = {null, null, ..., null}

Memory visualization :-

Action

Stack

Heap

int[] numbers

numbers $\rightarrow$ null

No memory allocated

numbers = new int[3]

number $\rightarrow$ [Address
 $\downarrow$
0x7a]

0x7a $\rightarrow$ [0, 0, 0]
(object allocated)

array input :-

```
int[] arr = new int[5]
```

```
for (int i = 0 ; i < arr.length ; i++) {
```

```
    arr[i] = in.nextInt();
```

```
}
```

Printing

```
for (int i = 0 ; i < arr.length ; i++) {
```

```
    System.out.print(arr[i] + " ");
```

```
}
```

using for each loop:

```
for (int num: arr) {  
    System.out.print(num);  
}
```

num $\Rightarrow$ Represents element of the array

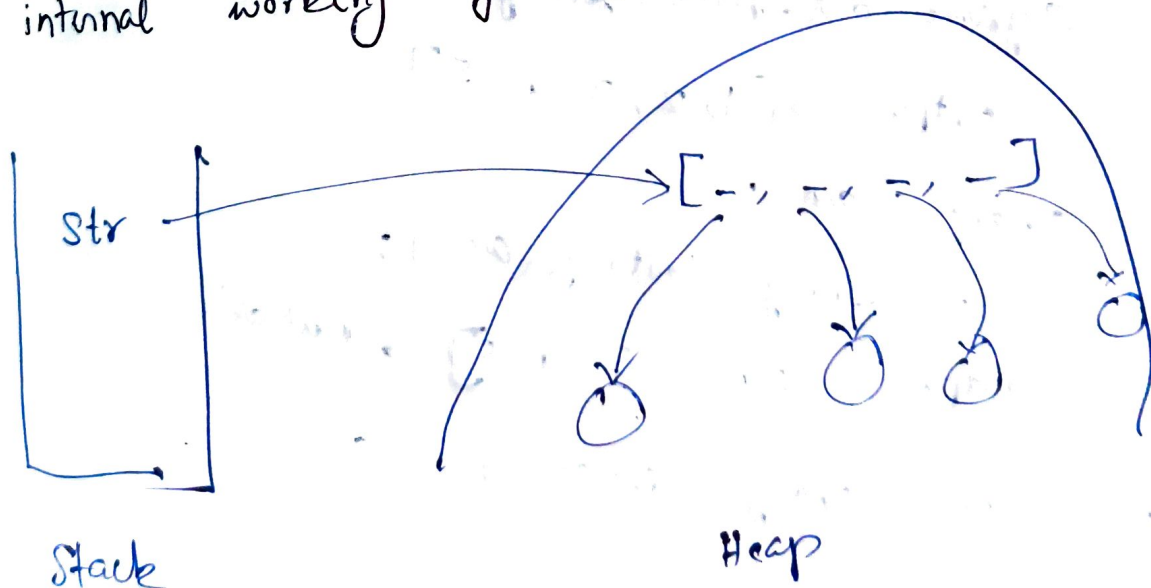
toString() method:

```
System.out.print(Arrays.toString(arr));
```

Array of objects:

```
String[] str = new String[4];
```

internal working of object



Features	primitive Arrays (e.g. int[])	Object Arrays (e.g. String[])
is the array itself an object	yes	yes
are the elements objects	No (They are primitive data types)	yes (They are references to objects)
Default value	0, 0.0, false	null

Passing array in a function:

 [1, 2, 3, 4, 5]
 int[] arr = new int[5];

System.out.print(Arrays.toString(arr));

output => [1, 2, 3, 4, 5]

change(arr);

System.out.print(Arrays.toString(arr));

output => [20, 2, 3, 4, 5] (mutable)

```
static void change(int[] arr) {
```

```
    arr[0] = 20;
```

```
}
```

2D Array :

looks like

1	2	3
4	5	6
7	8	9

Declaration :-

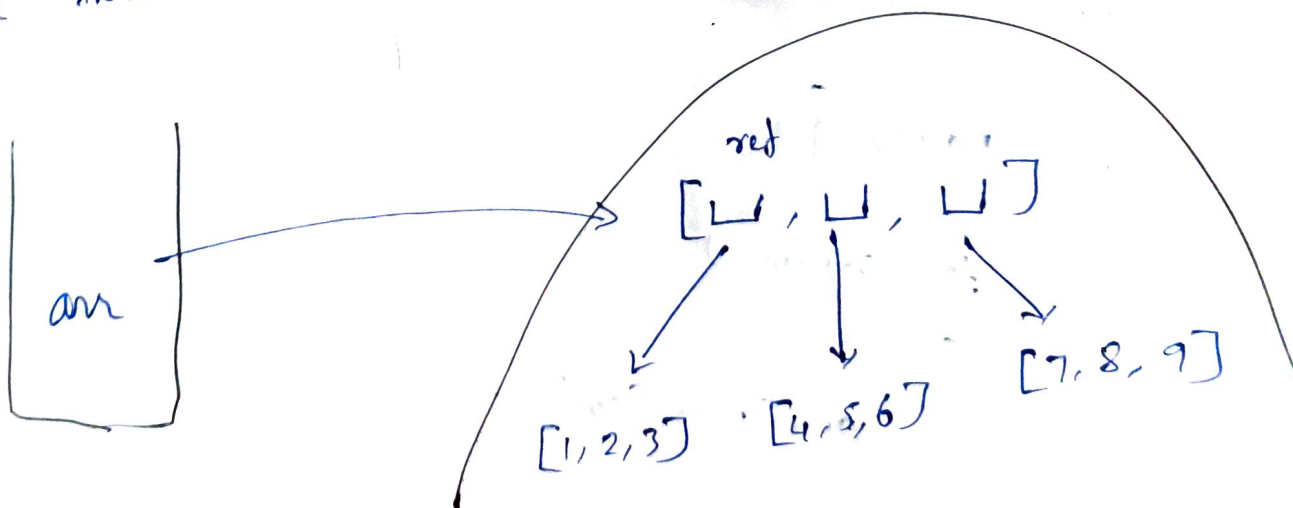
`int[][] arr;`

Initialization :-

`new int[3][3]`

Rows $\rightarrow 3$, columns $\rightarrow 3$

* A 2D array is an array whose elements are themselves arrays.



`arr[1]  $\rightarrow$  [4, 5, 6]`

`arr[1][0]  $\rightarrow$  4`

20) Jagged array :

```
int[][] arr = { {1, 2},  
                {3, 4, 5},  
                {6}  
            };
```

* Different row sizes.

* It is called jagged array.

Input for 2D Array :

Need nested loops.

Outer $\rightarrow$ rows

Inner $\rightarrow$ columns

```
for (rows) {
```

```
    for (columns)
```

```
    {
```

```
        arr[row][col] = ...
```

```
    }
```

```
}
```

arr.length $\Rightarrow$ no. of rows.

Output for 2D Array:

i) Nested loops

(or)

(ii) `Arrays.toString(arr[row]);`

for each row.

(iii) for each loop:-

```
for (int[] a : arr) {
```

```
    System.out.println(Arrays.toString(a));
```

Array List:

```
ArrayList<Integer> list = new ArrayList<>();
```

Adding element:

```
list.add();
```

```
list.get();
```

```
list.remove();
```